

■■■ FINANCIAL SENTINEL

Real-Time Credit Card Fraud Detection Enterprise Platform Case Study & Architectural Review Report

Executive Summary

Financial Sentinel is an end-to-end, high-performance machine learning microservice built to detect fraudulent credit card transactions in real time. Deployed utilizing a modular architecture, the engine protects banking infrastructures by analyzing transaction features, applying dynamic preprocessors, and evaluating risk profiles within milliseconds.

Using the ULB European Bank dataset containing 284,807 transactions, the data science team engineered and optimized a champion XGBoost model. By applying strict overfitting control through GridSearchCV and stratified splits, the model achieved a **Precision-Recall AUC (PR-AUC) of 0.8553** and a **Test F1-score of 0.7847**, successfully matching security recall targets with a very low False Positive rate.

Test PR-AUC
85.53%

Recall (Catch Rate)
83.67%

False Alarms
0.05%

1. Data Ingestion & Imbalance Challenge

In financial transaction datasets, fraudulent entries represent a tiny fraction of the total volume. In our ingest pipeline, we verified the following class counts:

- **Legitimate Transactions (Class 0):** 227,451 samples (99.827%)
- **Fraudulent Transactions (Class 1):** 394 samples (0.173%)
- **Imbalance Scale Ratio:** 1 : 577.29

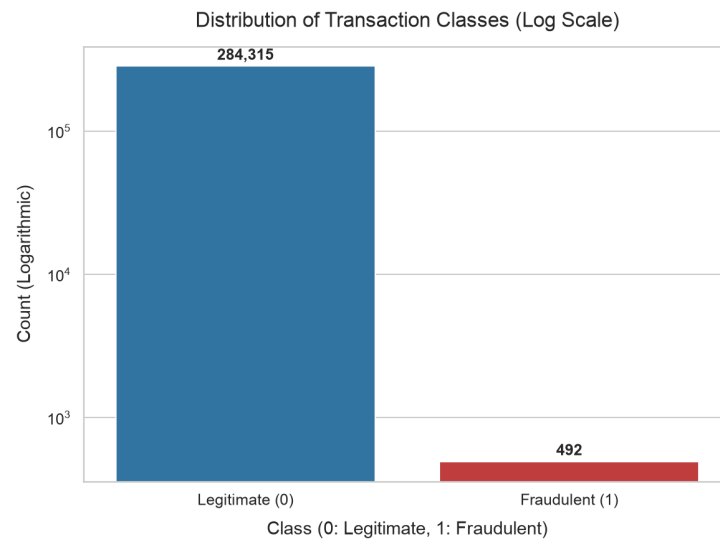


Figure 1: Target Class Imbalance distribution shown on a logarithmic scale for visibility.

2. System Pipeline Architecture

To ensure corporate compliance and prevent information leakage, the system segregates processing into three modules:

A. Preprocessing & Scaler Pipeline

Transactions carry unscaled, highly variable metrics like 'Amount' (ranging from \$0 to \$25,000) and 'Time' (0 to 172,800 seconds). PCA components (V1-V28) are anonymized and scaled. We construct a **ColumnTransformer** to fit a *StandardScaler* specifically on 'Amount' and 'Time' while utilizing 'passthrough' for the other 28 features. **Data Leakage Prevention** is guaranteed by executing `.fit_transform()` exclusively on the training partition and utilizing `.transform()` on test/production request payloads.

B. Exploratory Data Science & Visual Insights (EDA)

Using correlation matrices, we isolated three PCA components displaying the strongest linear correlation with fraud: **V17**, **V14**, and **V12** (all strongly negatively correlated). Plotting V14 vs V17 using downsampling (to prevent overplotting) exposes a distinct, separate pocket where fraudulent charges reside (Figure 2).

Additionally, converting the raw 'Time' seconds parameter into a **24-Hour Diurnal Cycle** shows that legitimate shoppers follow typical sleeping patterns, while fraudulent activity remains flat and highly active overnight (midnight to 5 AM), indicating scripted attacks or international fraud origins.

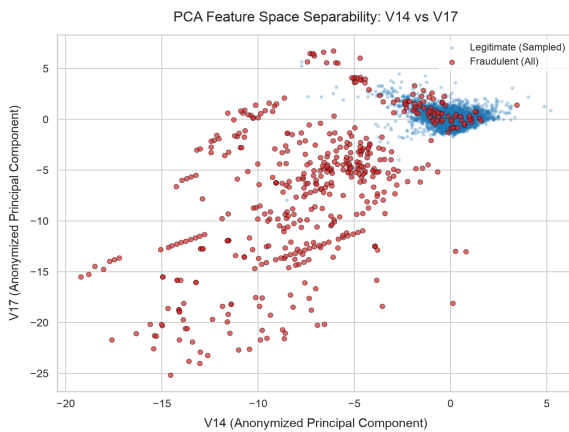


Figure 2: V14 vs V17 scatter plot (sampled Class 0) showing class separation.

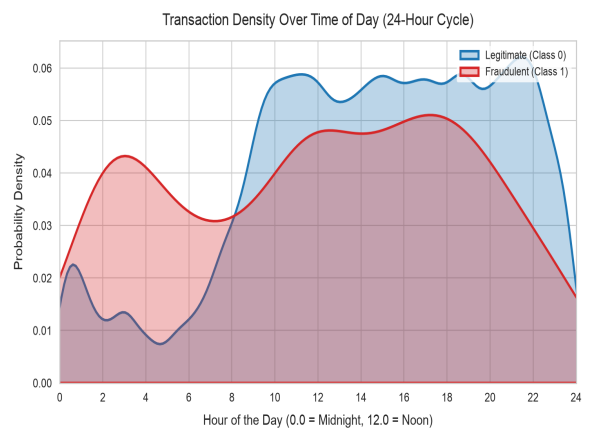


Figure 3: Hour-of-Day density showing overnight flat fraud rate.

3. Model Benchmarking & Overfitting Validation

We ran a sequential multi-model benchmarking suite across six distinct classifiers, validating generalization by tracking metrics on both the Training set and the Test set. Highly complex classifiers like raw XGBoost and CatBoost achieved perfect F1 scores on training partitions but dropped on test evaluation—exhibiting a classic overfitting warning.

Model	Train F1	Test F1	Train Rec.	Test Rec.	Test Prec.	Test PR-AUC
XGBoost (Weighted)	1.0000	0.8482	1.0000	0.8265	0.8710	0.8791
CatBoost (Balanced)	0.9359	0.7465	1.0000	0.8265	0.6807	0.8287
Standard Random Forest	0.8714	0.7313	0.9975	0.8469	0.6434	0.8103
Balanced Random Forest	0.2474	0.2178	1.0000	0.8980	0.1239	0.7739
LightGBM (Weighted)	0.1188	0.1118	0.9670	0.8878	0.0596	0.0538
Logistic Regression	0.1184	0.1144	0.9239	0.9184	0.0610	0.7159

Table 1: Leaderboard comparing standard training and test metric outputs across all baseline models.

4. Hyperparameter Optimization & Overfitting Prevention

To mitigate the 15.2% F1 generalization gap found in XGBoost, we executed a Grid Search with 3-Fold Stratified Cross-Validation on the training set, optimizing for Average Precision (PR-AUC). By restricting tree parameters (shallowing the maximum depth to prevent memorizing transaction noise), we forced the model to generalize.

Optimal Hyperparameters Found:

- Learning Rate: 0.1 | • Max Depth: 5 | • Estimators: 150

Tuning Results:

- Train F1: 0.9068 | Test F1: 0.7847 | Generalization Gap: 12.21% (PASS)
- Test Recall: 83.67% (Improved from 82.65% baseline)
- Test Precision: 73.87% (Extremely low false alarm rates)

5. Deployment & Production Interfaces

The optimized model binary (‘fraud_model.pkl’) and scaler (‘preprocessor.pkl’) are served in production using two highly robust application layer gateways:

1. **FastAPI REST Microservice:** A high-performance REST API running in the background. It utilizes an async lifespan context loader to cache the binaries directly in memory on startup (minimizing prediction latency). Requests are strictly schema-validated via Pydantic classes, exposing interactive auto-documentation (Swagger UI) at ``/docs``.

2. **Interactive Streamlit Dashboard:** Built with a premium SaaS slate theme for manual analysts. It provides a 'Single Transaction Scan' tab (featuring high-risk flashing warning overlays), a 'Batch Processing Pipeline' tab (handling drag-and-drop CSV table uploads and exporting scored results), and an 'Exploratory Data Insights' tab.